

MODULE GÉNIE LOGICIEL

Projet Intégratif — Refonte & Refactoring

Proposé Par Dr. Sana Ben Abdallah

2^{ème} année Cycle Ingénieur INLOG & INREV

Contexte et objectifs

Pour maîtriser et mettre en pratique les concepts, les bonnes pratiques et les outils explorés dans le cadre du module Génie Logiciel, l'application sur un projet réel collaboratif s'avère nécessaire. Les mêmes équipes ayant travaillé sur les différents thèmes d'exposés travailleront sur ce projet qui comptera comme examen.

Il vous est demandé de réaliser la refonte et le refactoring du PFE effectué en licence (choisir le PFE de l'un des membres de votre équipe). Chaque équipe a un chef de projet responsable entre autres de la coordination et du rendu final sur Classroom (archive .zip au nom du chef l'équipe).

Objectifs pédagogiques :

- Maîtrise par la pratique des thématiques abordées dans le cadre du cours.
- Cross-Évaluation / review des travaux de vos pairs de façon responsable et professionnelle.
- Auto-évaluation.
- Exploration et Réflexion sur les avancées de l'IA en génie logiciel.

Questions du projet

Question 1 — Spécification & Diagramme de classes initial [Tâche commune]

Donner une spécification informelle et semi-formelle des fonctionnalités principales de votre logiciel (2 pages maximum). Fournir également le diagramme de classes initial.

Cette tâche est commune à tous les membres de l'équipe.

Question 2 — Patrons GoF de création, de structure et de comportement

Appliquer **au moins** un patron de conception **de chacune** des trois catégories GoF, avec exemples concrets d'application dans le projet.

Chaque membre de l'équipe doit prendre en charge au moins UN patron GoF.

Question 3 — Patrons GRASP

Appliquer au moins un patron de conception GRASP par membre de l'équipe, avec exemples concrets d'application dans le projet.

Au moins 1 patron GRASP par membre.

Question 4 — Principes SOLID

Appliquer au moins un principe SOLID par membre de l'équipe, avec exemples concrets d'application dans le projet.

Au moins 1 principe SOLID par membre.

Question 5 — Contraintes OCL

Appliquer au moins une contrainte OCL par membre de l'équipe sur votre diagramme de classes.

Le niveau de difficulté de la contrainte est pris en compte dans la note attribuée.

Question 6 — Base de données [Tâche commune — approche comparative]

Cette question compare une approche traditionnelle et une approche assistée par IA pour la génération du modèle de base de données à partir du diagramme de classes.

a. Approche classique (sans IA) :

- Générer un modèle conceptuel de données (MCD) à partir de votre diagramme de classes en utilisant un outil non-IA de votre choix (ex. : Power AMC Designer, ou équivalent).
- Identifier les générations automatiques qui ne semblent pas conformes à la théorie et/ou qui diffèrent de la base de données implémentée dans votre PFE.

b. Approche IA :

Proposer la même génération avec un outil basé sur l'IA de votre choix (ex. : ChatGPT, GitHub Copilot, un LLM avec prompt dédié, etc.).

- Décrire votre démarche : outil(s) utilisé(s), prompt(s) formulé(s) (donner le ou les prompts exacts), stratégie adoptée.
- Comparer les résultats obtenus avec ceux de l'outil non-IA et avec la base de données de votre PFE.
- Quelles conclusions vous pouvez tirer / votre avis personnel ?

Question 7 — Génération de code [Tâche commune — approche comparative]

Cette question compare une approche traditionnelle et une approche assistée par IA pour la génération du squelette de code orienté objet à partir du diagramme de classes. Une attention particulière est portée aux relations de composition, d'agrégation et d'héritage.

a. Approche classique (sans IA) :

- Générer un code orienté objet (squelette de classes) à partir de votre diagramme de classes en utilisant un outil non-IA de votre choix.
- Identifier les générations automatiques qui ne semblent pas conformes à la théorie (notamment pour la composition, l'agrégation et l'héritage). Le résultat diffère-t-il du code implémenté dans votre PFE ?

b. Approche IA :

Proposer la même génération avec un outil basé sur l'IA de votre choix.

- Décrire votre démarche : outil(s) choisi(s), prompt(s) exact(s) utilisé(s), itérations éventuelles (prompt engineering).

- Évaluer la qualité du code généré pour les relations : composition, agrégation et héritage — ces points font souvent l'objet d'erreurs de génération automatique.
- Comparer avec le résultat de l'outil non-IA et avec le code original du PFE.
- Quelles conclusions vous pouvez tirer / votre avis personnel ?

Question 8 — IA & Software 2.0 dans votre projet [Question ouverte]

Choisissez l'une des deux options suivantes :

Option A — Bilan rétrospectif : Pour chaque question traitée (Q2 à Q7), indiquez **si et comment vous avez utilisé l'IA**. Précisez les outils, les prompts, les résultats obtenus et vos remarques critiques sur la qualité de l'aide apportée.

Option B — Exploration prospective : Pour chaque question du projet, proposez comment l'IA (ou une approche Software 2.0) pourrait être utilisée. Testez au moins une de vos propositions et comparez le résultat avec votre approche initiale sans IA.

Quelle que soit l'option choisie, votre analyse doit couvrir :

- Les outils et/ou modèles d'IA utilisés (ou envisagés).
- Les prompts formulés et la démarche de prompt engineering adoptée.
- Les résultats obtenus et leur évaluation critique (pertinence, exactitude, limites).
- Une comparaison explicite : approche traditionnelle vs. approche IA.
- **Votre position personnelle : à quelles étapes du cycle de vie logiciel l'IA apporte-t-elle le plus de valeur dans votre contexte ?**

Consignes générales

Pour chaque patron GOF ou GRASP, principe SOLID ou contrainte OCL (Q2 à Q5), il faut :

1. Donner un exemple tiré d'une référence fiable (citer la référence).
2. Suivre l'exemple pour appliquer le patron / principe / contrainte dans le cas de votre projet.
3. Identifier clairement les composantes du patron dans votre cas (ex. : quelle classe joue le rôle d'adaptateur, de Singleton, de Fabrique...) en argumentant votre choix.

Traçabilité des responsabilités

Indiquer clairement, pour chaque tâche, le nom de l'étudiant responsable (les notes sont individuelles). Exemple : « Patron Adaptateur (Prénom Nom) ».

Une liste récapitulative Quoi / Qui doit figurer en fin de rapport.

Rôle du responsable d'équipe

- Coordonner tout le travail et l'attribution des tâches entre membres.
- Envoyer le travail de son équipe (voir section Livrables attendus) — la date et la modalité d'envoi vous seront communiquées ultérieurement.
- Être le porte-parole de son équipe le jour de la validation et répartir équitablement le temps alloué entre les différents membres.

Livrables attendus : UNE unique archive .zip au nom du chef l'équipe

#	Livrable	Détail
1	Rapport	Q1, Q6 et Q7 : rendu commun. Q2 à Q5 et Q8 : partagé entre les membres (au moins 1 tâche/membre/question).
2	Lien Git	Tous les codes d'implémentation. Chaque membre dépose le code dont il est responsable.
3	Vidéos explicatives	Au moins une vidéo commentée en voix off par membre (Q2 à Q8), présentant l'utilisation des outils et/ou les implémentations réalisées.
4	Rapports de cross-évaluation	Deux rapports de cross-évaluation des deux équipes de son sous-groupe
5	Rapport d'auto-évaluation.	Un rapport d'auto-évaluation de l'équipe.

Critères d'évaluation (indicatifs)

Critère	Pondération indicative
Spécification & diagramme de classes initial (Q1)	Commun — socle du projet
Patrons GoF correctement appliqués et argumentés (Q2)	Individuel — qualité > quantité
Patrons GRASP (Q3)	Individuel
Principes SOLID (Q4)	Individuel
Contraintes OCL (Q5, difficulté prise en compte)	Individuel
Génération BD — comparaison outil classique vs. IA (Q6)	Commun + analyse critique
Génération code — comparaison outil classique vs. IA (Q7)	Commun + analyse critique
IA & Software 2.0 — profondeur de l'analyse (Q8)	Individuel
Qualité du rapport, des vidéos et du dépôt Git	Collectif
Maitrise et qualité de l'évaluation (des pairs ou auto-évaluation)	Individuel

Modalités d'évaluation

Cross-évaluation entre équipes

- Chaque groupe (Grp1 et Grp2) sera réparti en deux sous-groupes, chacun constitué par 3 équipes de projet différentes. Ces trois équipes feront une cross-évaluation des travaux du sous-groupe (chaque équipe évaluera donc le travail de deux autres équipes : évaluation deux à deux).
- L'évaluation se fera en deux temps : une évaluation à mi-chemin et une évaluation finale
- Pour la validation à mi-chemin, un passage de 45 mn est consacré à chaque sous-groupe. Chaque équipe sera évaluée en 15 mn : **10 mn** d'évaluation par les deux autres équipes (par ses pairs) + **5 mn** d'auto-évaluation.
- Pour la validation finale, un passage de 27 mn est consacré à chaque sous-groupe. Chaque équipe sera évaluée en 9 mn : **6 mn** d'évaluation par les deux autres équipes (par ses pairs) + **3 mn** d'auto-évaluation.

Dates de dépôt et de validation

#	Date	Échéance
1 dépôt pour la validation à mi-chemin	Avril	Date limite de remise des travaux entre équipes (pour la cross-évaluation). La modalité de dépôt et la répartition des sous-groupes de cross-évaluation seront communiquées ultérieurement - dates exactes à fixer ASAP.
2 validation à mi-chemin	Avril	Pré-soutenance et cross-évaluation - dates exactes à fixer ASAP.
3 dépôt pour la validation finale	À communiquer ultérieurement	Le dépôt se fera à travers un devoir Classroom. Date à communiquer également sur classroom.
4 validation finale	Voir calendrier des examens publié par la direction des études	Soutenance finale — programmée par l'administration (voir calendrier des examens). La planification de l'ordre de passage sera communiquée sur Classroom.

Bon courage à toutes les équipes !